SOFTWARE PROJECT REPORT · 2026

Student Management System

A role-based platform for everyday school operations

Project explanation · System design · Data model · Workflows · Security · Setup

Prepared by	**Class / section**
_____	_____
Submitted to	**Institution**
_____	_____

Prepared from the project repository · 17 October 2026

Java 21 · Spring Boot · Spring Security · Thymeleaf · JPA · SQLite

01 · PROJECT AT A GLANCE

Executive summary

The Student Management System (SMS) is a web application that organises school administration and learning workflows in one role-based portal. It provides separate experiences for administrators, teachers, and students, backed by shared, linked school records.

The system brings together student and staff profiles, class and subject assignments, attendance, grades, assignments, announcements, reports, and student documents. After sign-in, each user lands in the portal for their role and can access only permitted records and actions.

3 user roles	Connected records	PDF outputs
Administrator, teacher, student	Accounts, classes, subjects, academic activity	Report card and student ID card

Problem addressed

School information held in separate registers and files is harder to keep consistent, retrieve, and share. SMS provides a central place to maintain records and supports the daily flow between administrators, teachers, and learners.

Project objectives

- Maintain structured school users, student/teacher profiles, classes, and subjects.
- Enable teachers to record attendance and marks and manage class assignments.
- Let students review their own school information and submit assignment work.
- Give administrators school management and reporting screens.
- Protect credentials and enforce role and record ownership boundaries.

> **Project in one sentence:** One Spring Boot application connects administration, teaching, and student self-service through role-based access.

Scope

This is a server-rendered application suited to a local school project or demonstration. SQLite and local file storage make setup simple; it is not presented as a cloud-hosted, multi-school enterprise system.

Roles and modules

Role	Responsibilities	Outcome
Administrator	Manage student and teacher accounts, classes, and subjects; publish announcements; open reports; generate student documents.	Organised school directory and oversight.
Teacher	Work with assigned classes/subjects; view rosters; mark attendance; enter grades; create assignments; review submissions; share updates; view reports.	Teaching activity and progress recorded centrally.
Student	View profile, attendance, grades, assignments, and announcements; submit work; download report and ID cards; change own password.	Self-service access to personal school information.

Accounts and access

Username or email sign-in, role-specific landing pages, BCrypt password hashes.

School directory

People, classes, subjects, class rosters, and teacher-to-class/subject assignments.

Academic records

Daily attendance and marks by subject and exam type, with summaries for learners and staff.

Assignments

Teachers create class work and due dates; students submit files; teachers review submissions.

Announcements

School-wide or class-targeted messages, shown to the intended audience.

Reports and PDFs

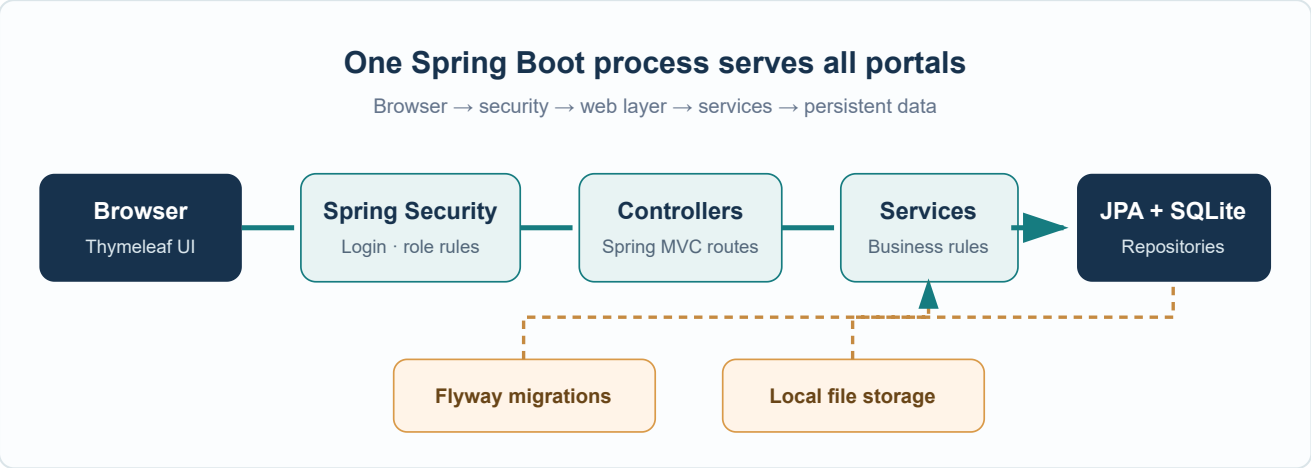
Attendance/performance summaries, A4 report cards, and student ID cards.

> **Example:** A teacher records attendance for a class and date. One saved row per student feeds the student's attendance view and report summary.

03 · APPLICATION ARCHITECTURE

How a request moves through SMS

The application uses a layered Spring MVC design. Controllers receive web requests, services apply workflow rules, repositories persist entities through JPA, and Thymeleaf renders HTML back to the browser.



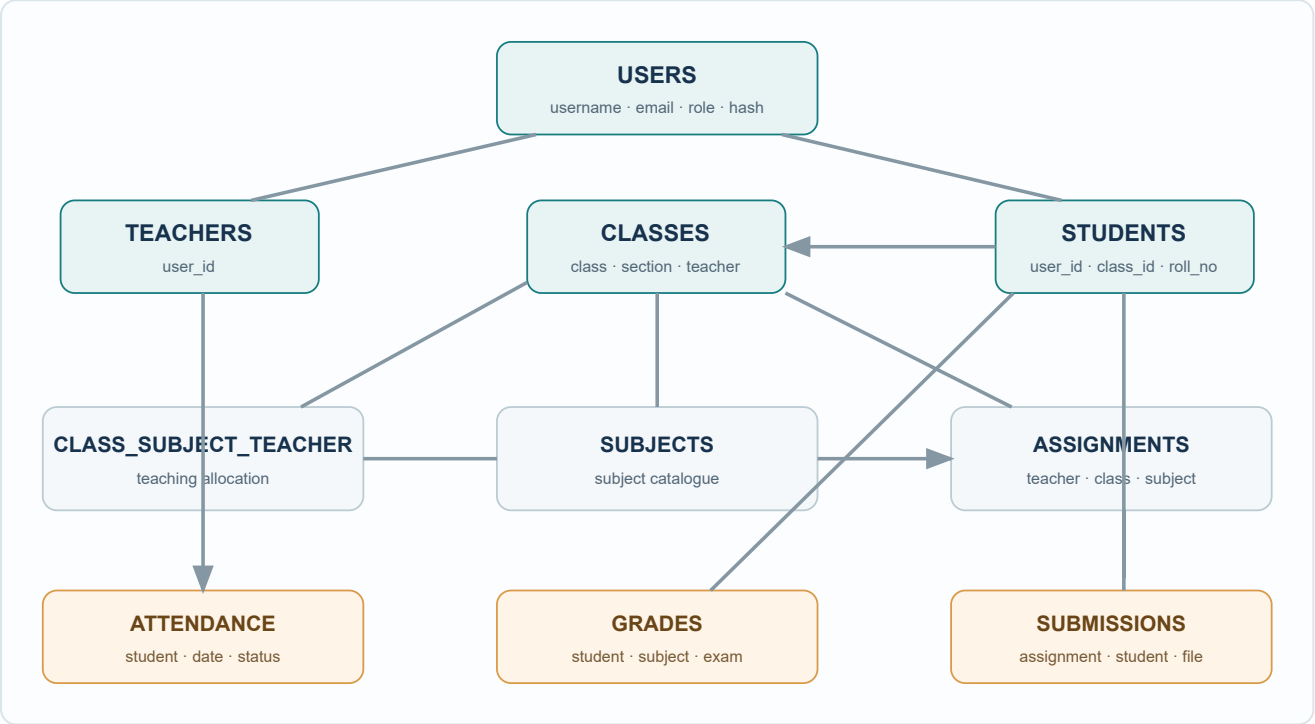
Layer	Responsibility	Project examples
Presentation	Pages and form input	Thymeleaf templates, Bootstrap
Web/security	Route actions and protect requests	Controllers, SecurityConfig, role handler
Service	Apply use cases, ownership rules, calculations	Portal services, ReportService
Persistence	Map entities to relational records	Spring Data repositories, Hibernate/JPA, SQLite
Schema/files	Version tables and store attachments	Flyway SQL, FileStorage

Architecture choice: SMS is a modular monolith: features are separated into packages but run as one Spring Boot application. This keeps local operation and explanation straightforward.

04 · DATABASE DESIGN

Entities and relationships

The user account is separate from student and teacher profile data. Academic records reference the people, class, subject, or assignment that they belong to. Foreign keys and unique constraints protect relationships and prevent duplicates.



Integrity rules in the schema

- Usernames and emails are unique; each student or teacher profile links to one user.
- Class name/section is unique; a roll number is unique within its class.
- Attendance is unique per student/day; grades are unique per student/subject/exam.
- One submission is allowed per student/assignment. Foreign keys prevent orphan records.

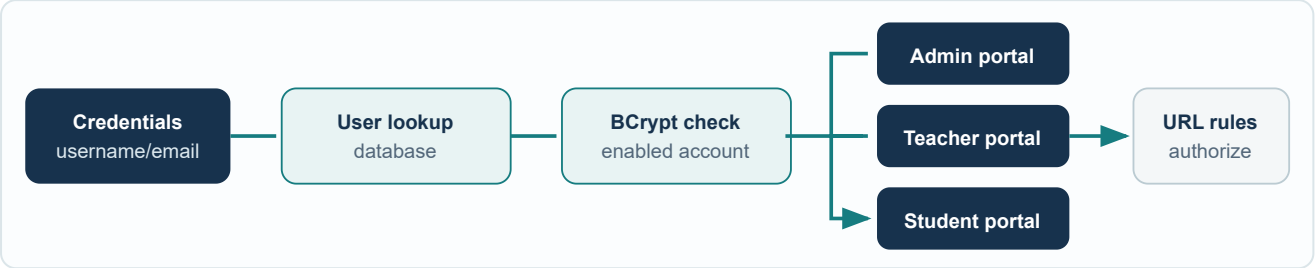
Flyway V1 creates the core schema. V2 adds the original upload filename used when a student's submitted file is downloaded.

> **Announcements:** each announcement links to its author and may optionally target a class; no target class means school-wide.

05 · CORE WORKFLOWS

From user action to saved result

Login and role routing



Spring Security loads accounts by username or email, checks the supplied password against the stored BCrypt hash, then redirects to the dashboard for the authenticated role. Route rules protect later requests.

Attendance and grades

1. Teacher opens an assigned class (and subject for grades).
2. Service verifies the teacher's class-teacher or subject-teacher assignment.
3. Validated records are saved; unique constraints reject duplicates.
4. Student views and report services summarize the saved records.

Assignment submission

Teacher creates class work and a due date. A student accesses assignments for their class and uploads a file. The app checks presence, size and extension, stores it with a random UUID name, and saves the database relationship. Download routes check that the current student owns the submission or the teacher has relevant assignment access.

Announcements and reports

Announcements can be school-wide or class-targeted. Report services aggregate attendance and grade information. OpenPDF formats an A4 report card or a student ID card at standard ID-1 dimensions.

> **Common path:** Form → controller → service validation and access check → repository/entity → SQLite → page or PDF.

Protecting data and validating input

Authentication

Form login accepts username/email. BCrypt stores one-way password hashes rather than plain-text passwords.

Role authorization

Spring Security restricts `/admin/**`, `/teacher/**`, and `/student/**` to matching roles.

Ownership checks

Services resolve the current person and verify class, subject, assignment, and submission access.

Form validation

Application checks cover required fields, uniqueness, password length (minimum 8), marks, dates, and workflow conditions.

Upload limits

Maximum 10 MB; accepted: PDF, DOC/DOCX, TXT, JPG/JPEG, PNG, ZIP.

File/path safety

Random stored names and normalized paths keep user filenames out of storage paths; downloads go through access-controlled routes.

Consistency and rollback behavior

Foreign keys and uniqueness rules provide a second layer of data integrity. FileStorage removes a newly stored file if the surrounding database transaction rolls back and coordinates deletion with transaction completion.

Automated tests in the repository

Test classes cover application security, administrator management, announcements, reports, student portal, and teacher portal/assignment flows. MockMvc checks public login, role-specific dashboard access, forbidden cross-role access, and redirects. The existence of tests is not a substitute for demonstrating each feature in a running browser.

> **Defense-ready explanation:** “Authentication establishes the user’s identity. Authorization checks the role and, for school data, the specific class or record the user is allowed to access.”

Data protection boundary

SQLite and local uploads are convenient for local demonstration. A real deployment would need operational planning for backups, access to the host, database/storage protection, monitoring, and recovery.

Technology choices and code map

Technology	Use in the project
Java 21 / Spring Boot 4.1.1	Application language, runtime, configuration
Spring MVC + Thymeleaf	Web routes, forms, server-rendered pages
Spring Security	Form login, role authorization, BCrypt
Spring Data JPA / Hibernate	Entity mapping and persistence
SQLite + Flyway	Local relational database and versioned schema
OpenPDF	Report-card and ID-card PDF generation
Bootstrap 5	Responsive layout and form components

Package / folder	Responsibility
`auth`	Users, roles, account service, login, admin seeding.
`admin`, `teacher`, `student`	Role-specific controllers, services, and forms.
`school`	Core school entities and repositories.
`report`, `common`, `announcement`	Reports/PDFs; security/files/errors; announcement model and service.
`resources/templates`	Portal pages and shared Thymeleaf fragments.
`resources/db/migration`	Flyway SQL schema versions.
`src/test/java`	Automated application and feature tests.

Run, demonstrate, and explain

Run locally

1. Install Java 21 and open the project root containing `pom.xml`.
2. Set the first admin password before initial startup:

```
$env:SMS_ADMIN_PASSWORD = "choose-a-strong-password"
```

3. Run in PowerShell:

```
.\mvnw.cmd spring-boot:run
```

4. Open `http://localhost:8080/login`. Flyway prepares the schema; the first admin is created only if no administrator exists.

Defaults are `./sms.db` and `./uploads`. Environment variables can configure database URL, upload folder, school name, admin credentials, and demo seeding. Demo data is enabled by default for local development; use `SMS_DEMO_DATA_ENABLED=false` when not wanted and change the demo password before sharing a seeded instance.

Suggested teacher demonstration

1. Admin: show management of a student, teacher, or class.
2. Teacher: show a roster, attendance/grades, and assignment creation.
3. Student: show personal records, announcements, and assignment submission.
4. Generate a report/ID card and explain how access is controlled.

Closing summary

SMS connects school records and academic workflows in a maintainable, role-based Spring application. Its core design ensures each action is linked to the correct user, class, and subject, with application-level checks supported by relational constraints.

Future extensions (not current features): server database and cloud file storage, automated backup/monitoring, and additional analytics as deployment needs grow.